VetCare

Sistema de clínica veterinária

Documentação técnica para projeto de Banco de Dados

SQLite · Python · Flask
Modelagem, implementação e scripts comentados

Versão 1.0
Outubro de 2026

Sumário

1. Introdução 3

2. Objetivo geral 3

3. Objetivos específicos 3

4. Escopo e requisitos 4

5. Regras de negócio 5

6. Modelo conceitual 6

7. Modelo lógico 7

8. Normalização e decisões de modelagem 8

9. Modelo físico e dicionário de dados 9

10. Explicação do script SQL 12

11. Arquitetura e organização do código 14

12. API e transações 15

13. Interface e uso 15

14. Instalação e operação 16

15. Verificação e limites 16

16. Referências técnicas 17

17. Interface da aplicação 18

Apêndice 1. 01_schema.sql 19

Apêndice 2. 02_seed.sql 21

Apêndice 3. 03_queries.sql 22

Apêndice 4. 04_transactions.sql 23

Documentação técnica para projeto de Banco de Dados. Versão 1.0, outubro de 2026.

1. Introdução

Uma clínica veterinária precisa relacionar o animal atendido ao seu responsável, ao profissional que realiza a consulta e ao serviço prestado. Agenda, prontuário e recebimento pertencem ao mesmo processo, mas registram fatos diferentes. O horário de uma consulta pode mudar antes do atendimento; o prontuário descreve o que foi observado; o pagamento informa quanto foi recebido e quando.

O VetCare organiza esse processo em uma aplicação web local. O projeto usa SQLite para apresentar a passagem do modelo de dados ao banco físico, com restrições, consultas e transações que podem ser examinadas diretamente. A interface permite executar as operações sem exigir comandos SQL do usuário da clínica.

Os registros iniciais são fictícios e servem à apresentação. O projeto não pretende substituir um sistema clínico em produção. Seu escopo é uma clínica, sem estoque, contas de usuário, faturamento fiscal ou estornos.

2. Objetivo geral

Desenvolver e documentar um sistema de clínica veterinária que mantenha os cadastros, a agenda, os prontuários e os recebimentos em um banco relacional SQLite, com uma interface web para operar e demonstrar as regras do modelo.

3. Objetivos específicos

- Identificar as entidades do atendimento e definir suas relações e cardinalidades.
- Implementar tabelas com chaves primárias, chaves estrangeiras, limites de domínio e unicidade.
- Separar cadastro, atendimento e pagamento, evitando duplicação de dados descritivos.
- Impedir sobreposição de consultas para o mesmo veterinário ou animal.
- Preservar preço e duração da consulta quando o catálogo de serviços mudar.
- Concluir a consulta e criar seu prontuário em uma única transação.
- Registrar pagamentos parciais e calcular o saldo sem armazenar um total redundante.
- Demonstrar JOIN, LEFT JOIN, agregações, índices e operações de manipulação de dados.
- Documentar a instalação, o código, o script SQL e os testes que verificam as regras.

4. Escopo e requisitos

A recepção cadastra os responsáveis e os animais e organiza a agenda. O veterinário registra o atendimento. O financeiro registra recebimentos. Esses papéis descrevem o uso do sistema; a versão local não contém autenticação nem restrições por perfil.

Código	Requisito funcional	Tela ou operação
RF01	Cadastrar, editar, pesquisar, arquivar e reativar tutores	Tutores
RF02	Cadastrar animais vinculados a um tutor e consultar seu histórico	Pacientes
RF03	Manter veterinários com CRMV único	Veterinários
RF04	Manter serviços, preço e duração	Serviços
RF05	Agendar, reagendar, confirmar e cancelar consultas abertas	Agenda
RF06	Bloquear conflitos de horário para veterinário ou animal	Trigger de agenda
RF07	Registrar queixa, avaliação, conduta e peso no prontuário	Atendimento
RF08	Concluir consulta junto com o registro clínico	Transação de atendimento
RF09	Registrar pagamentos parciais sem ultrapassar o saldo	Financeiro
RF10	Exibir agenda do dia, pacientes ativos, receita mensal e pendências	Visão geral
RF11	Preservar o histórico ao arquivar cadastros	Cadastros

Código	Requisito não funcional	Implementação
RNF01	Instalação sem servidor externo de banco	Arquivo SQLite local
RNF02	Persistência entre reinícios	Banco na pasta instance
RNF03	Interface utilizável em desktop e celular	Layout responsivo e tabelas com rolagem
RNF04	Operações atômicas no atendimento	Transação SQLite com rollback em falhas
RNF05	Entradas validadas e consultas parametrizadas	validation.py e parâmetros SQL
RNF06	Navegação por teclado e identificação dos campos	HTML semântico, rótulos e foco visível
RNF07	Histórico preservado	FK RESTRICT e triggers de imutabilidade
RNF08	Código e banco verificáveis	Testes com banco temporário e scripts independentes

5. Regras de negócio

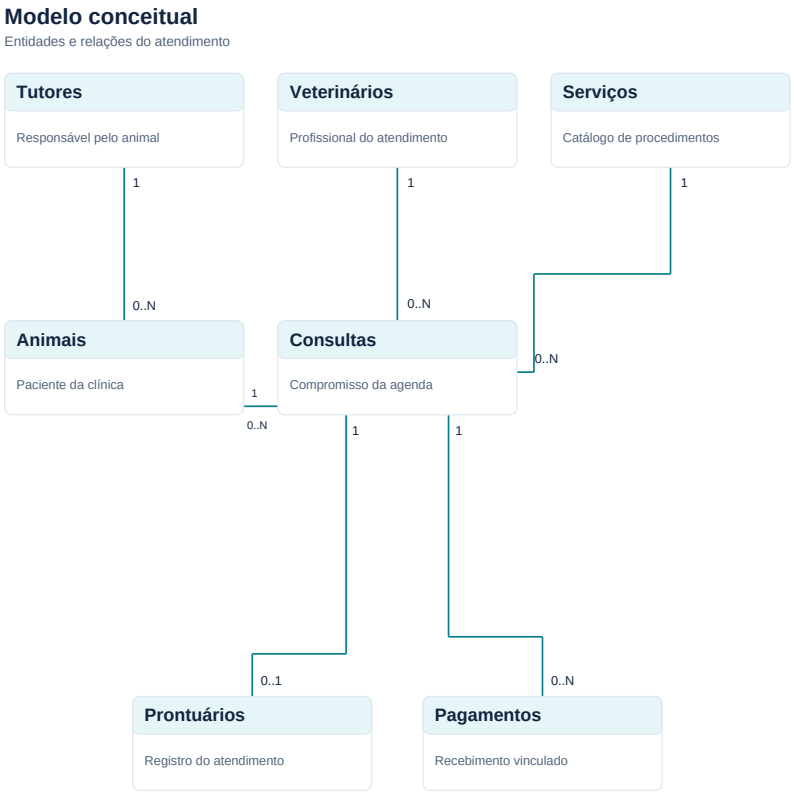
Código	Regra	Onde é aplicada
RN01	Cada animal pertence a exatamente um tutor	pets.tutor_id NOT NULL e FK
RN02	Uma consulta referencia um animal, um veterinário e um serviço	FKs obrigatórias de appointments
RN03	CRMV e nome de serviço são únicos	UNIQUE
RN04	Consultas não canceladas do mesmo profissional ou animal não se sobrepõem	Triggers de INSERT e UPDATE
RN05	Horários adjacentes são permitidos	Comparação de intervalos semiabertos
RN06	Uma consulta tem no máximo um prontuário	medical_records.appointment_id UNIQUE
RN07	A aplicação conclui o atendimento junto com a criação do prontuário	Transação em finish_appointment
RN08	Um recebimento deve ser positivo e respeitar o saldo	CHECK e trigger payment_limit
RN09	Uma consulta com recebimentos não pode ser cancelada	Trigger appointment_finance_guard
RN10	Preço e duração da consulta conservam o valor contratado	Snapshots em appointments
RN11	Prontuários e pagamentos não podem ser editados ou removidos	Triggers de imutabilidade
RN12	Cadastro arquivado não entra em novo agendamento	Validação da API
RN13	Não se arquiva cadastro com consultas abertas; tutor exige animais arquivados	Validação da API
RN14	Animal com histórico não troca de tutor nesta versão	Validação da API
RN15	Nascimento e pagamento não podem ter data futura	Validação da API
RN16	A aplicação não conclui consultas de uma data futura	Validação da API

As regras de domínio fixas ficam no banco sempre que o SQLite permite expressá-las de modo direto. Regras de fluxo e de calendário ficam na API. Executar SQL manualmente pode contornar as regras exclusivas da API; os scripts didáticos devem ser executados em um banco de demonstração.

O preço zero é permitido para retornos gratuitos. O campo em aberto do painel soma apenas saldos de consultas concluídas. A tela financeira também mostra consultas agendadas com saldo, pois o sistema permite adiantamentos.

6. Modelo conceitual

As entidades do domínio são Tutor, Animal, Veterinário, Serviço, Consulta, Prontuário e Pagamento. Tutor identifica o responsável pelo animal. Consulta representa o compromisso da agenda e liga as entidades do atendimento. Prontuário e Pagamento registram resultados distintos dessa consulta.

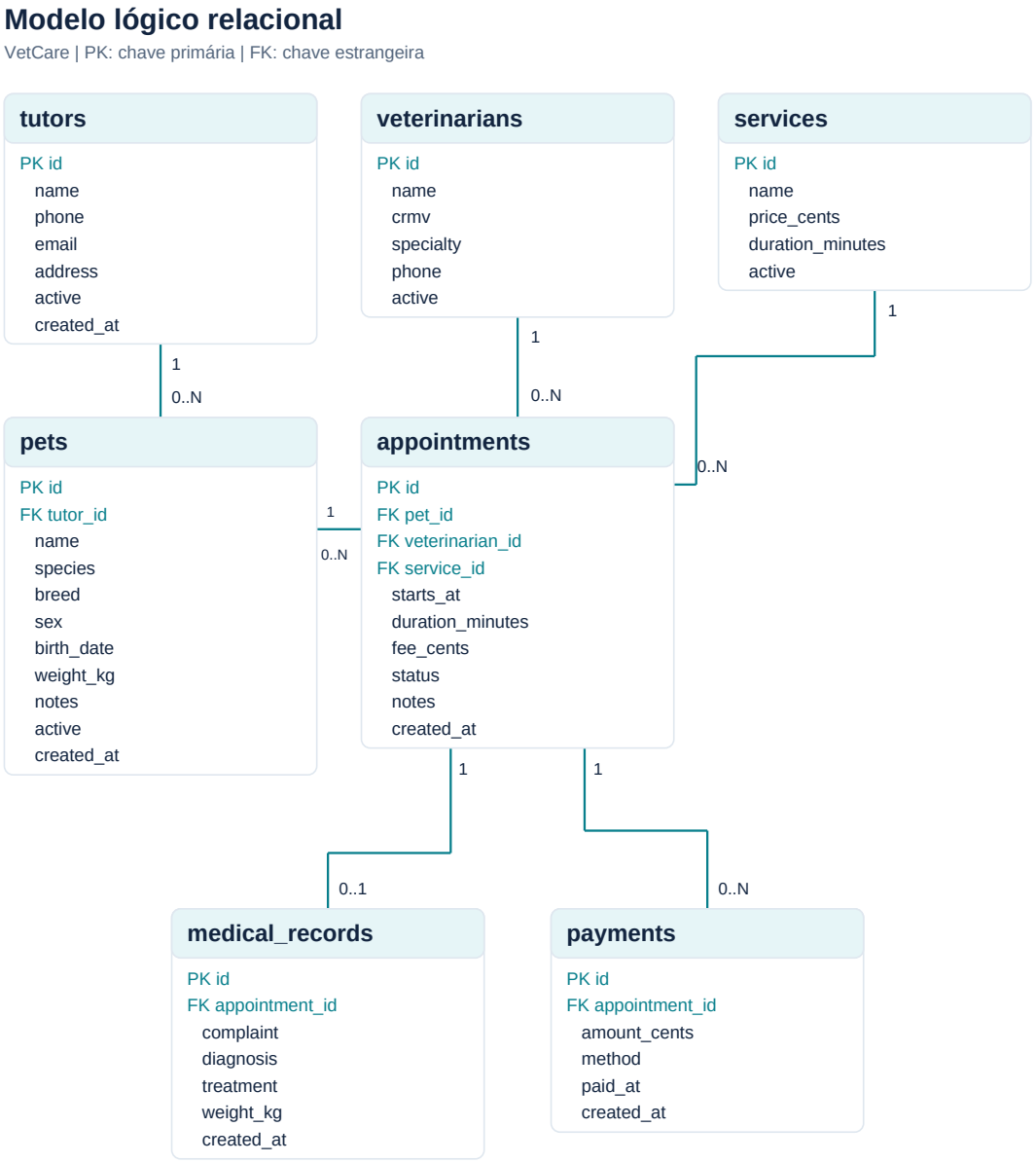


Relação	Cardinalidade	Interpretação
Tutor possui Animal	Tutor 1 : Animal 0..N	Tutor pode não ter animais; cada animal exige um tutor
Animal participa de Consulta	Animal 1 : Consulta 0..N	Cada consulta atende um animal
Veterinário realiza Consulta	Veterinário 1 : Consulta 0..N	Cada consulta tem um profissional
Serviço compõe Consulta	Serviço 1 : Consulta 0..N	Uma consulta usa um serviço neste escopo
Consulta gera Prontuário	Consulta 1 : Prontuário 0..1	Consulta aberta ou cancelada pode não ter prontuário
Consulta recebe Pagamento	Consulta 1 : Pagamento 0..N	Consulta pode não ter recebido ou ter vários pagamentos

A relação entre consulta e serviço foi limitada a um serviço por consulta para manter o projeto pequeno. Uma consulta com vários procedimentos exigiria uma entidade associativa, como itens da consulta, com quantidade e preço de cada item.

7. Modelo lógico

O modelo lógico usa tabelas com identificadores inteiros. Os nomes de tabelas e colunas estão em inglês para manter consistência com o código. Os rótulos da interface estão em português.



- tutors(id PK, name, phone, email, address, active, created_at)
- pets(id PK, tutor_id FK, name, species, breed, sex, birth_date, weight_kg, notes, active, created_at)
- veterinarians(id PK, name, crmv UNIQUE, specialty, phone, active)
- services(id PK, name UNIQUE, price_cents, duration_minutes, active)
- appointments(id PK, pet_id FK, veterinarian_id FK, service_id FK, starts_at, duration_minutes, fee_cents, status, notes, created_at)
- medical_records(id PK, appointment_id FK UNIQUE, complaint, diagnosis, treatment, weight_kg, created_at)
- payments(id PK, appointment_id FK, amount_cents, method, paid_at, created_at)

As FKs usam ON DELETE RESTRICT. A interface arquiva tutores, animais, veterinários e serviços por meio de active=0. Isso preserva vínculos e evita excluir dados usados em consultas anteriores.

8. Normalização e decisões de modelagem

Na primeira forma normal, cada coluna guarda um valor do seu domínio. Pagamentos e prontuários são linhas próprias; não há uma lista de consultas ou recebimentos dentro de uma coluna. Endereço é um texto de exibição, sem necessidade de consultas por logradouro neste escopo.

As tabelas usam chaves simples. Assim, não há dependências parciais de uma chave composta, atendendo à segunda forma normal. CRMV é uma chave candidata de veterinarians e name é uma chave candidata de services, ambos com UNIQUE.

Na terceira forma normal, nomes do tutor, do veterinário e do serviço não são copiados para appointments. Eles são obtidos por JOIN. O prontuário referencia a consulta, que já informa animal e profissional. Pagamento referencia a consulta, que informa a cobrança.

fee_cents e duration_minutes de appointments representam condições da consulta no momento do agendamento. Não são cópias que precisam permanecer iguais ao catálogo: mudar o preço de services não muda o preço já acordado. Essa decisão preserva a interpretação histórica dos dados.

weight_kg em pets informa o peso mais recente registrado pela interface. weight_kg em medical_records registra a medição daquele atendimento. A transação de conclusão atualiza ambos quando um peso é informado. Os campos têm significados temporais distintos; o peso histórico não é substituído quando o cadastro muda.

O total recebido e o saldo são calculados pela view appointment_balances. A view agrega pagamentos antes de juntá-los à consulta, produzindo uma linha por consulta. Isso evita somar o preço da consulta várias vezes quando existem pagamentos parciais.

9. Modelo físico e dicionário de dados

O script 01_schema.sql cria sete tabelas STRICT, sete índices explícitos, dez triggers e uma view. Também existem índices internos associados a UNIQUE. Tabelas STRICT exigem SQLite 3.37 ou superior [1]. INTEGER armazena IDs, flags e centavos; TEXT armazena textos e datas; REAL armazena peso.

A seção seguinte é gerada a partir do esquema real com PRAGMA table_info e PRAGMA foreign_key_list. As expressões CHECK e os detalhes dos triggers estão no script completo do apêndice.

9.1. Tutores (tutors)

Campo	Tipo	Obrigatório	Chave / padrão	Descrição
id	INTEGER	Sim	PK	Identificador único gerado pelo SQLite.
name	TEXT	Sim	Sem padrão	Nome de exibição do cadastro.
phone	TEXT	Sim	Sem padrão	Telefone para contato, armazenado como texto.
email	TEXT	Não	Sem padrão	E-mail opcional.
address	TEXT	Sim	Padrão: ''	Endereço de exibição do tutor.
active	INTEGER	Sim	Padrão: 1	1: ativo; 0: arquivado.
created_at	TEXT	Sim	Padrão: UTC automático	Instante de criação em UTC, com sufixo Z.

9.2. Animais (pets)

Campo	Tipo	Obrigatório	Chave / padrão	Descrição
id	INTEGER	Sim	PK	Identificador único gerado pelo SQLite.
tutor_id	INTEGER	Sim	FK: tutors.id	Tutor responsável pelo animal.
name	TEXT	Sim	Sem padrão	Nome de exibição do cadastro.
species	TEXT	Sim	Sem padrão	Cão, Gato, Ave ou Outro.
breed	TEXT	Sim	Padrão: ''	Raça informada, sem catálogo auxiliar.
sex	TEXT	Sim	Padrão: 'Não informado'	Macho, Fêmea ou Não informado.
birth_date	TEXT	Não	Sem padrão	Nascimento opcional em YYYY-MM-DD.
weight_kg	REAL	Não	Sem padrão	Peso atual do cadastro; atualizado pelo último registro da interface.
notes	TEXT	Sim	Padrão: ''	Observações opcionais do cadastro ou agendamento.
active	INTEGER	Sim	Padrão: 1	1: ativo; 0: arquivado.
created_at	TEXT	Sim	Padrão: UTC automático	Instante de criação em UTC, com sufixo Z.

9.3. Veterinários (veterinarians)

Campo	Tipo	Obrigatório	Chave / padrão	Descrição
id	INTEGER	Sim	PK	Identificador único gerado pelo SQLite.
name	TEXT	Sim	Sem padrão	Nome de exibição do cadastro.
crmv	TEXT	Sim	UNIQUE	Registro profissional com UF; único.
specialty	TEXT	Sim	Padrão: 'Clínica geral'	Área de atuação informada.
phone	TEXT	Sim	Padrão: ''	Telefone para contato, armazenado como texto.
active	INTEGER	Sim	Padrão: 1	1: ativo; 0: arquivado.

9.4. Serviços (services)

Campo	Tipo	Obrigatório	Chave / padrão	Descrição
id	INTEGER	Sim	PK	Identificador único gerado pelo SQLite.
name	TEXT	Sim	UNIQUE	Nome de exibição do cadastro.
price_cents	INTEGER	Sim	Sem padrão	Preço atual do serviço em centavos, não negativo.
duration_minutes	INTEGER	Sim	Padrão: 30	Duração em minutos, de 10 a 240.
active	INTEGER	Sim	Padrão: 1	1: ativo; 0: arquivado.

9.5. Consultas (appointments)

Campo	Tipo	Obrigatório	Chave / padrão	Descrição
id	INTEGER	Sim	PK	Identificador único gerado pelo SQLite.
pet_id	INTEGER	Sim	FK: pets.id	Animal da consulta.
veterinarian_id	INTEGER	Sim	FK: veterinarians.id	Profissional da consulta.
service_id	INTEGER	Sim	FK: services.id	Serviço da consulta.
starts_at	TEXT	Sim	Sem padrão	Início em YYYY-MM-DDTHH:MM, horário de São Paulo.
duration_minutes	INTEGER	Sim	Sem padrão	Duração em minutos, de 10 a 240.
fee_cents	INTEGER	Sim	Sem padrão	Preço acordado em centavos; preservado após alterar o catálogo.
status	TEXT	Sim	Padrão: 'Agendado'	Agendado, Confirmado, Concluído ou Cancelado.
notes	TEXT	Sim	Padrão: ''	Observações opcionais do cadastro ou agendamento.
created_at	TEXT	Sim	Padrão: UTC automático	Instante de criação em UTC, com sufixo Z.

9.6. Prontuários (medical_records)

Campo	Tipo	Obrigatório	Chave / padrão	Descrição
id	INTEGER	Sim	PK	Identificador único gerado pelo SQLite.
appointment_id	INTEGER	Sim	FK: appointments.id; UNIQUE	Consulta vinculada ao registro.
complaint	TEXT	Sim	Sem padrão	Queixa ou motivo do atendimento.
diagnosis	TEXT	Sim	Sem padrão	Avaliação clínica ou diagnóstico registrado.
treatment	TEXT	Sim	Padrão: ''	Conduta e orientações registradas.
weight_kg	REAL	Não	Sem padrão	Peso aferido neste atendimento; registro histórico.
created_at	TEXT	Sim	Padrão: UTC automático	Instante de criação em UTC, com sufixo Z.

9.7. Pagamentos (payments)

Campo	Tipo	Obrigatório	Chave / padrão	Descrição
id	INTEGER	Sim	PK	Identificador único gerado pelo SQLite.
appointment_id	INTEGER	Sim	FK: appointments.id	Consulta vinculada ao registro.
amount_cents	INTEGER	Sim	Sem padrão	Valor recebido em centavos; deve ser positivo.
method	TEXT	Sim	Sem padrão	Pix, Dinheiro ou Cartão.
paid_at	TEXT	Sim	Sem padrão	Dia do pagamento em YYYY-MM-DD.
created_at	TEXT	Sim	Padrão: UTC automático	Instante de criação em UTC, com sufixo Z.

10. Explicação do script SQL

10.1. Inicialização e integridade referencial

01_schema.sql ativa foreign_keys antes da transação, cria os objetos com IF NOT EXISTS e grava user_version=1. A execução é não destrutiva. IF NOT EXISTS não é um mecanismo de migração: alterações futuras do esquema precisam de um script de migração próprio.

O SQLite exige que a aplicação ative a verificação de chaves estrangeiras em cada conexão [2]. get_db faz isso ao abrir o banco. A API também configura um tempo de espera de dez segundos para bloqueios de escrita. A inicialização habilita WAL, e o comando de backup usa a API de backup do SQLite.

10.2. Tipos, valores e datas

Os valores monetários usam inteiros em centavos. R$ 180,00 é armazenado como 18000. A entrada passa por Decimal antes de ser convertida; valores com mais de duas casas decimais são rejeitados. Isso evita operações de saldo com aproximações binárias de ponto flutuante.

A agenda usa texto no formato YYYY-MM-DDTHH:MM, interpretado como horário civil de America/Sao_Paulo. Nascimento e pagamento usam YYYY-MM-DD. created_at usa UTC com sufixo Z para registrar o instante de criação. Datas de calendário são validadas na API e por CHECK, usando a normalização do SQLite para rejeitar dias inexistentes.

clock.py define o dia operacional pela zona America/Sao_Paulo. Nos scripts de demonstração, date('now','-3 hours') usa a regra UTC-3 vigente para São Paulo. Se a regra de fuso mudar, esses scripts precisarão ser ajustados.

10.3. Conflitos de agenda

Os triggers appointment_no_overlap_insert e appointment_no_overlap_update verificam registros existentes para o mesmo veterinário ou animal [3]. O conflito ocorre quando o início de A é anterior ao fim de B e o início de B é anterior ao fim de A. Consultas canceladas não ocupam horário.

Uma consulta das 09:00 às 09:30 permite outra às 09:30. Uma consulta às 09:20, com duração de 30 minutos, é bloqueada. No UPDATE, o registro que está sendo alterado é excluído da comparação pelo seu ID.

10.4. Histórico e financeiro

appointment_history_guard bloqueia alterações em consultas que já têm prontuário. record_completed_only exige que a consulta esteja concluída antes de inserir o registro clínico. A aplicação atualiza o status e insere o prontuário na mesma transação, evitando uma conclusão incompleta pelo fluxo da interface.

payment_limit compara o novo valor com a soma já recebida e o preço da consulta. Se a soma ultrapassa o preço ou a consulta está cancelada, o INSERT é abortado. appointment_finance_guard impede cancelamento com recebimentos, troca de animal, profissional ou serviço após receber e redução do preço abaixo do que já foi pago.

Os quatro triggers de imutabilidade impedem UPDATE e DELETE em pagamentos e prontuários. Correções clínicas e estornos exigiriam registros adicionais com justificativa e autoria, que não fazem parte desta versão. Esse limite foi mantido visível nos formulários.

10.5. Índices e view

idx_pets_tutor apoia a busca de animais por responsável. Os índices de agenda combinam veterinário ou animal com horário, ajudando a limitar a busca de conflitos. idx_agenda_service apoia o vínculo com o catálogo, e idx_payments_appointment apoia a agregação de recebimentos. idx_agenda_date permite buscas por intervalo de data.

Uma consulta que aplica substr diretamente em starts_at pode não aproveitar o índice por data para a busca. O exemplo Q8 mostra uma busca por intervalo e permite observar seu plano com EXPLAIN QUERY PLAN. Para uma base pequena de apresentação, algumas listagens usam substr por clareza; consultas de produção deveriam ser ajustadas ao volume medido.

appointment_balances é uma view, não uma tabela duplicada. Ela apresenta as colunas da consulta, o valor recebido e o saldo. COALESCE converte a ausência de pagamentos em zero.

10.6. Carga, consultas e transações

02_seed.sql carrega seis tutores, oito animais, dois veterinários, quatro serviços, nove consultas, dois prontuários e dois recebimentos. O código de inicialização recusa a carga se alguma tabela já tiver dados. As datas relativas mantêm a agenda útil em apresentações posteriores.

03_queries.sql contém oito exemplos: agenda do dia, quantidade de animais por tutor, receita por forma de pagamento, saldos, histórico clínico, volume de atendimentos, verificações de integridade e plano de consulta. Eles mostram JOIN, LEFT JOIN, GROUP BY, SUM, COUNT e COALESCE.

04_transactions.sql demonstra INSERT, UPDATE e DELETE dentro de transações que terminam em ROLLBACK. O primeiro exemplo cria tutor e animal relacionados; o segundo altera o catálogo sem mudar consultas anteriores; o terceiro remove apenas registros temporários sem dependências restantes.

11. Arquitetura e organização do código

O navegador carrega templates/index.html e os arquivos estáticos. As telas usam a API Flask na mesma origem. A API valida os campos, executa SQL parametrizado e devolve JSON. O SQLite confirma ou rejeita a operação conforme as restrições do esquema.

Arquivo	Responsabilidade
app.py	Cria a aplicação, configura sessão, CSRF, cabeçalhos e comandos de banco
api.py	Implementa cadastros, consultas, prontuários, pagamentos e listagem inicial
db.py	Abre uma conexão por requisição, fecha conexões e inicializa o banco
validation.py	Valida texto, inteiros, opções, peso, dinheiro e datas
clock.py	Define a data operacional da clínica
static/app.js	Controla navegação, eventos e atualização do estado
static/views.js	Monta as telas a partir dos dados da API
static/forms.js	Monta formulários com rótulos e limites
static/client.js	Faz as requisições e trata erros HTTP
static/ui.js	Reúne ícones, tabelas, modais e escape de texto
static/style.css	Define cores, tipografia, espaçamento e comportamento responsivo
tests/test_system.py	Testa operações e regras em bancos temporários
scripts/build_docs.py	Gera diagramas, dicionário e relatório

O frontend foi dividido por responsabilidade para que a navegação, os formulários e a apresentação não dependam de um único arquivo extenso. Não há etapa de build. O navegador executa módulos JavaScript nativos.

Os nomes de tabela interpolados em SQL pertencem a uma whitelist interna. Valores recebidos do usuário entram como parâmetros. Textos exibidos na interface passam por escapeHTML antes de serem inseridos no HTML.

12. API e transações

Método e rota	Operação	Resultado
GET /api/bootstrap	Carrega cadastros, consultas, recebimentos e resumo	200 com JSON
POST /api/tutors, pets, veterinarians ou services	Cria cadastro	201 com ID
PATCH /api/{cadastro}/{id}	Edita cadastro ou altera active	200
POST /api/appointments	Agenda consulta com preço e duração do catálogo	201 com ID
PATCH /api/appointments/{id}	Reagenda ou confirma/cancela consulta aberta	200
GET /api/pets/{id}/history	Lista consultas e prontuários do animal	200 com JSON
POST /api/appointments/{id}/record	Conclui atendimento e grava prontuário	201
POST /api/payments	Registra recebimento parcial ou total	201 com ID

Validação de conteúdo devolve 400. Ausência de token de sessão devolve 403. Conflitos de integridade devolvem 409. Indisponibilidade do banco devolve 503, com a falha registrada no servidor. A interface apresenta a mensagem junto ao formulário e permite corrigir o campo.

A conclusão do atendimento usa BEGIN IMMEDIATE. O UPDATE exige um status aberto, e o INSERT exige uma consulta concluída. Se uma dessas operações falha, o contexto de transação executa rollback. O peso atual do animal é atualizado na mesma transação quando o prontuário informa uma medição.

Os recebimentos são serializados pelo bloqueio de escrita do SQLite. O trigger verifica o saldo no momento do INSERT, impedindo que dois pagamentos sucessivos ultrapassem a cobrança. O banco foi escolhido para demonstração local com poucos usuários; não foi realizado ensaio de carga para uso simultâneo em produção.

13. Interface e uso

A visão geral exibe as consultas do dia, o número de pacientes ativos, os recebimentos do mês e os saldos das consultas concluídas. A agenda oferece data, busca textual e status. Os cadastros mostram registros ativos por padrão, com opção de incluir arquivados.

A consulta abre um resumo com responsável, telefone, serviço, horário, valor e saldo. A partir desse resumo, o usuário pode confirmar, cancelar, atender, receber ou abrir o histórico do animal. A interface informa que prontuários e recebimentos serão preservados após o registro.

Os formulários usam rótulos, campos obrigatórios, limites numéricos e mensagens de erro. Os modais usam dialog nativo, com navegação por teclado e fechamento por Escape. Em telas pequenas, a navegação passa

para uma faixa horizontal; tabelas mantêm suas colunas em uma área com rolagem, sem reduzir os textos a tamanhos ilegíveis.

14. Instalação e operação

Requisitos: Python 3.11 ou superior, SQLite 3.37 ou superior e navegador com suporte a módulos JavaScript e dialog. Flask é a dependência do servidor; tzdata fornece a base de fusos em sistemas sem esses arquivos, como algumas instalações do Windows. ReportLab só é necessário para gerar o PDF novamente [4].

A sequência de instalação é criar um ambiente virtual, instalar requirements.txt, executar init-db e iniciar app.py. O README contém os comandos para Windows, Linux e macOS. init-db --demo carrega os dados fictícios, enquanto init-db cria apenas o esquema.

O banco padrão fica em instance/vetcare.sqlite3. VETCARE_DATABASE permite escolher outro arquivo. VETCARE_SECRET define uma chave de sessão estável; sem essa variável, a chave é gerada no início e sessões anteriores deixam de valer após reiniciar o servidor. PORT altera a porta, cujo padrão é 5000.

backup-db cria uma cópia consistente pela API SQLite e recusa sobrescrever um destino existente. Copiar apenas o arquivo principal enquanto o servidor escreve pode perder dados ainda no WAL; por isso o projeto fornece um comando de backup [5]. Para restaurar, pare o servidor e configure o arquivo de backup como banco de execução.

A versão entregue roda localmente e escuta somente em 127.0.0.1. Uma função da Vercel não oferece o arquivo permanente compartilhado necessário para esse SQLite local [6]. Uma publicação online exige armazenamento persistente no backend ou migração para um serviço SQLite remoto. Essa implantação não integra o escopo local.

15. Verificação e limites

A suíte de integração cobre o cadastro ao pagamento parcial, conflito por veterinário, conflito por animal, horários adjacentes, reagendamento, liberação de horário após cancelamento, cobrança acima do saldo, preservação do preço antigo e proteção do histórico. Também verifica CSRF, JSON inválido, unicidade de CRMV, precisão monetária, datas inexistentes e inicialização sem perda dos dados.

Os testes usam bancos temporários separados do arquivo da demonstração. As verificações de integridade usam PRAGMA integrity_check e foreign_key_check. A execução é descrita no README, e o resultado observado está em docs/verificacao.md.

O sistema carrega as listagens completas em /api/bootstrap. Paginação, filtros no servidor e revisão do plano de execução seriam necessários antes de trabalhar com grande volume. A busca atual é textual e não aplica um vocabulário clínico.

Não há login, auditoria por autor, prontuário assinado, envio de lembretes, emissão fiscal, controle de vacinas ou estoque. O cadastro guarda o tutor atual do animal; transferências de responsabilidade exigiriam um histórico próprio. A consulta inclui um serviço. Estornos e correções clínicas também exigiriam novas entidades e regras.

Esses limites definem o que a versão demonstra e o que deve ser modelado antes de ampliar seu uso.

16. Referências técnicas

[1] SQLite. STRICT tables. https://www.sqlite.org/stricttables.html. Acesso em 6 out. 2026.

[2] SQLite. Foreign key support. https://www.sqlite.org/foreignkeys.html. Acesso em 6 out. 2026.

[3] SQLite. CREATE TRIGGER. https://www.sqlite.org/lang_createtrigger.html. Acesso em 6 out. 2026.

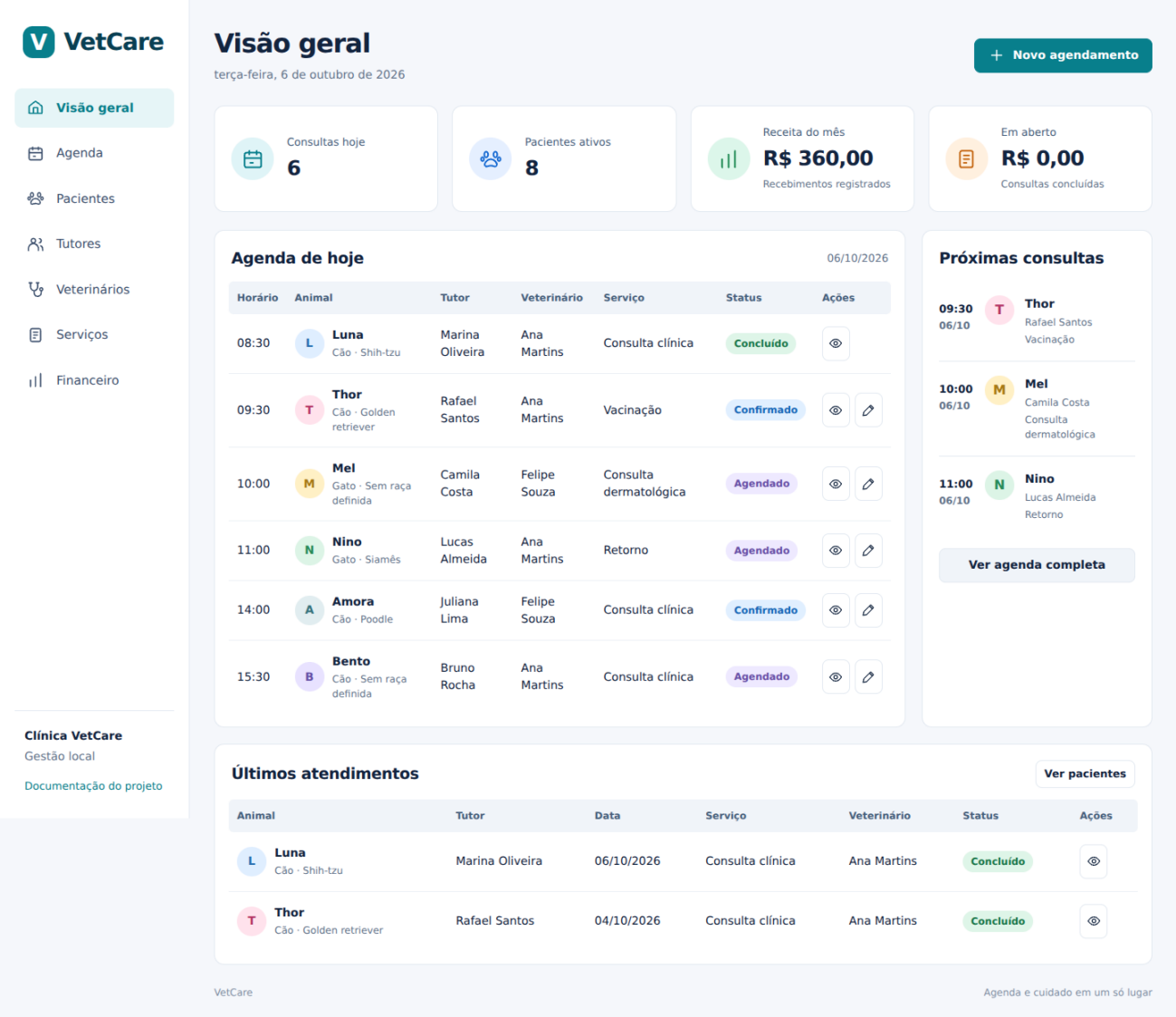
[4] Pallets Projects. Flask: installation. https://flask.palletsprojects.com/en/stable/installation/. Acesso em 6 out. 2026.

[5] Python Software Foundation. sqlite3: Connection.backup. https://docs.python.org/3/library/sqlite3.html#sqlite3.Connection.backup. Acesso em 6 out. 2026.

[6] Vercel. Is SQLite supported in Vercel? https://vercel.com/kb/guide/is-sqlite-supported-in-vercel. Acesso em 6 out. 2026.

17. Interface da aplicação

Visão geral com os dados fictícios do projeto.



Apêndice 1. 01_schema.sql

O arquivo no diretório database é a versão executável. Linhas longas têm quebra visual neste apêndice.

```
-- VetCare | SQLite 3.37+ | criação não destrutiva, esquema versão 1.
-- Cada conexão deve habilitar foreign_keys. Valores monetários são CENTAVOS.
PRAGMA foreign_keys = ON;
BEGIN;
CREATE TABLE IF NOT EXISTS tutors (
 id INTEGER PRIMARY KEY,
 name TEXT NOT NULL CHECK(length(trim(name)) BETWEEN 2 AND 100),
 phone TEXT NOT NULL CHECK(length(trim(phone)) BETWEEN 8 AND 25),
 email TEXT CHECK(email IS NULL OR length(email) <= 150),
 address TEXT NOT NULL DEFAULT '' CHECK(length(address) <= 250),
 active INTEGER NOT NULL DEFAULT 1 CHECK(active IN (0,1)),
 created_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ','now'))
) STRICT;
CREATE TABLE IF NOT EXISTS pets (
 id INTEGER PRIMARY KEY,
 tutor_id INTEGER NOT NULL REFERENCES tutors(id) ON DELETE RESTRICT,
 name TEXT NOT NULL CHECK(length(trim(name)) BETWEEN 1 AND 80),
 species TEXT NOT NULL CHECK(species IN ('Cão','Gato','Ave','Outro')),
 breed TEXT NOT NULL DEFAULT '' CHECK(length(breed) <= 80),
 sex TEXT NOT NULL DEFAULT 'Não informado' CHECK(sex IN ('Macho','Fêmea','Não informado')),
 birth_date TEXT CHECK(birth_date IS NULL OR (date(birth_date,'+0 days') IS NOT NULL AND date(birth_date,'+0 days') =
    birth_date)),
 weight_kg REAL CHECK(weight_kg IS NULL OR weight_kg > 0),
 notes TEXT NOT NULL DEFAULT '' CHECK(length(notes) <= 4000),
 active INTEGER NOT NULL DEFAULT 1 CHECK(active IN (0,1)),
 created_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ','now'))
) STRICT;
CREATE TABLE IF NOT EXISTS veterinarians (
 id INTEGER PRIMARY KEY,
 name TEXT NOT NULL CHECK(length(trim(name)) BETWEEN 2 AND 100),
 crmv TEXT NOT NULL UNIQUE CHECK(length(trim(crmv)) BETWEEN 3 AND 30),
 specialty TEXT NOT NULL DEFAULT 'Clínica geral' CHECK(length(specialty) <= 100),
 phone TEXT NOT NULL DEFAULT '' CHECK(length(phone) <= 25),
 active INTEGER NOT NULL DEFAULT 1 CHECK(active IN (0,1))
) STRICT;
CREATE TABLE IF NOT EXISTS services (
 id INTEGER PRIMARY KEY,
 name TEXT NOT NULL UNIQUE CHECK(length(trim(name)) BETWEEN 2 AND 100),
 price_cents INTEGER NOT NULL CHECK(price_cents >= 0),
 duration_minutes INTEGER NOT NULL DEFAULT 30 CHECK(duration_minutes BETWEEN 10 AND 240),
 active INTEGER NOT NULL DEFAULT 1 CHECK(active IN (0,1))
) STRICT;
-- Preço e duração são snapshots: alterações no catálogo não reescrevem o passado.
-- Datas da agenda usam horário civil America/Sao_Paulo: YYYY-MM-DDTHH:MM.
CREATE TABLE IF NOT EXISTS appointments (
 id INTEGER PRIMARY KEY,
 pet_id INTEGER NOT NULL REFERENCES pets(id) ON DELETE RESTRICT,
 veterinarian_id INTEGER NOT NULL REFERENCES veterinarians(id) ON DELETE RESTRICT,
 service_id INTEGER NOT NULL REFERENCES services(id) ON DELETE RESTRICT,
 starts_at TEXT NOT NULL CHECK(length(starts_at)=16 AND strftime('%Y-%m-%dT%H:%M',starts_at,'+0 days') IS NOT NULL AND
    strftime('%Y-%m-%dT%H:%M',starts_at,'+0 days')=starts_at),
 duration_minutes INTEGER NOT NULL CHECK(duration_minutes BETWEEN 10 AND 240),
 fee_cents INTEGER NOT NULL CHECK(fee_cents >= 0),
 status TEXT NOT NULL DEFAULT 'Agendado' CHECK(status IN ('Agendado','Confirmado','Concluído','Cancelado')),
 notes TEXT NOT NULL DEFAULT '' CHECK(length(notes) <= 4000),
 created_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ','now'))
) STRICT;
CREATE TABLE IF NOT EXISTS medical_records (
 id INTEGER PRIMARY KEY,
 appointment_id INTEGER NOT NULL UNIQUE REFERENCES appointments(id) ON DELETE RESTRICT,
 complaint TEXT NOT NULL CHECK(length(trim(complaint)) BETWEEN 2 AND 4000),
 diagnosis TEXT NOT NULL CHECK(length(trim(diagnosis)) BETWEEN 2 AND 4000),
 treatment TEXT NOT NULL DEFAULT '' CHECK(length(treatment) <= 4000),
 weight_kg REAL CHECK(weight_kg IS NULL OR weight_kg > 0),
 created_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ','now'))
) STRICT;
CREATE TABLE IF NOT EXISTS payments (
 id INTEGER PRIMARY KEY,
 appointment_id INTEGER NOT NULL REFERENCES appointments(id) ON DELETE RESTRICT,
 amount_cents INTEGER NOT NULL CHECK(amount_cents > 0),
 method TEXT NOT NULL CHECK(method IN ('Pix','Dinheiro','Cartão')),
 paid_at TEXT NOT NULL CHECK(date(paid_at,'+0 days') IS NOT NULL AND date(paid_at,'+0 days')=paid_at),
 created_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ','now'))
) STRICT;
```

```
-- Índices nas FKs e na agenda evitam varreduras nas consultas mais frequentes.
CREATE INDEX IF NOT EXISTS idx_pets_tutor ON pets(tutor_id);
CREATE INDEX IF NOT EXISTS idx_agenda_vet ON appointments(veterinarian_id,starts_at);
CREATE INDEX IF NOT EXISTS idx_agenda_pet ON appointments(pet_id,starts_at);
CREATE INDEX IF NOT EXISTS idx_agenda_service ON appointments(service_id);
CREATE INDEX IF NOT EXISTS idx_agenda_date ON appointments(starts_at,status);
CREATE INDEX IF NOT EXISTS idx_payments_appointment ON payments(appointment_id);
-- Intervalos semiabertos [início,fim): consultas consecutivas são permitidas.
CREATE TRIGGER IF NOT EXISTS appointment_no_overlap_insert BEFORE INSERT ON appointments
WHEN NEW.status <> 'Cancelado'
BEGIN
 SELECT RAISE(ABORT,'Conflito de horário: veterinário ou paciente ocupado.')
 WHERE EXISTS(SELECT 1 FROM appointments a WHERE a.status <> 'Cancelado'
 AND (a.veterinarian_id=NEW.veterinarian_id OR a.pet_id=NEW.pet_id)
 AND datetime(a.starts_at) < datetime(NEW.starts_at,'+' || NEW.duration_minutes || ' minutes')
 AND datetime(NEW.starts_at) < datetime(a.starts_at,'+' || a.duration_minutes || ' minutes'));
END;
CREATE TRIGGER IF NOT EXISTS appointment_no_overlap_update BEFORE UPDATE ON appointments
WHEN NEW.status <> 'Cancelado'
BEGIN
 SELECT RAISE(ABORT,'Conflito de horário: veterinário ou paciente ocupado.')
 WHERE EXISTS(SELECT 1 FROM appointments a WHERE a.id <> NEW.id AND a.status <> 'Cancelado'
 AND (a.veterinarian_id=NEW.veterinarian_id OR a.pet_id=NEW.pet_id)
 AND datetime(a.starts_at) < datetime(NEW.starts_at,'+' || NEW.duration_minutes || ' minutes')
 AND datetime(NEW.starts_at) < datetime(a.starts_at,'+' || a.duration_minutes || ' minutes'));
END;
CREATE TRIGGER IF NOT EXISTS appointment_history_guard BEFORE UPDATE ON appointments
WHEN EXISTS(SELECT 1 FROM medical_records WHERE appointment_id=OLD.id)
BEGIN SELECT RAISE(ABORT,'Consulta com prontuário não pode ser alterada.'); END;
CREATE TRIGGER IF NOT EXISTS appointment_finance_guard BEFORE UPDATE ON appointments
WHEN (NEW.status='Cancelado' OR NEW.pet_id<>OLD.pet_id OR NEW.veterinarian_id<>OLD.veterinarian_id OR
    NEW.service_id<>OLD.service_id OR NEW.fee_cents < (SELECT COALESCE(SUM(amount_cents),0) FROM payments WHERE
    appointment_id=OLD.id))
 AND EXISTS(SELECT 1 FROM payments WHERE appointment_id=OLD.id)
BEGIN SELECT RAISE(ABORT,'Consulta com recebimentos não pode trocar vínculos, ser cancelada ou ter valor inferior ao
    recebido.'); END;
CREATE TRIGGER IF NOT EXISTS record_completed_only BEFORE INSERT ON medical_records
BEGIN
 SELECT RAISE(ABORT,'Prontuário exige consulta concluída.')
 WHERE COALESCE((SELECT status FROM appointments WHERE id=NEW.appointment_id),'') <> 'Concluído';
END;
CREATE TRIGGER IF NOT EXISTS payment_limit BEFORE INSERT ON payments
BEGIN
 SELECT RAISE(ABORT,'Pagamento excede o saldo ou consulta está cancelada.')
 WHERE NOT EXISTS(SELECT 1 FROM appointments a WHERE a.id=NEW.appointment_id AND a.status <> 'Cancelado'
 AND NEW.amount_cents + (SELECT COALESCE(SUM(amount_cents),0) FROM payments WHERE appointment_id=a.id) <= a.fee_cents);
END;
-- Recebimentos e prontuários são append-only no escopo didático; estorno é evolução futura.
CREATE TRIGGER IF NOT EXISTS payment_no_update BEFORE UPDATE ON payments
BEGIN SELECT RAISE(ABORT,'Recebimentos não podem ser editados.'); END;
CREATE TRIGGER IF NOT EXISTS payment_no_delete BEFORE DELETE ON payments
BEGIN SELECT RAISE(ABORT,'Recebimentos não podem ser removidos.'); END;
CREATE TRIGGER IF NOT EXISTS record_no_update BEFORE UPDATE ON medical_records
BEGIN SELECT RAISE(ABORT,'Prontuários não podem ser editados.'); END;
CREATE TRIGGER IF NOT EXISTS record_no_delete BEFORE DELETE ON medical_records
BEGIN SELECT RAISE(ABORT,'Prontuários não podem ser removidos.'); END;
-- A view calcula saldo, evitando armazenar um total derivado sujeito a inconsistências.
CREATE VIEW IF NOT EXISTS appointment_balances AS
 SELECT a.*, COALESCE(p.received_cents,0) AS received_cents,
 a.fee_cents - COALESCE(p.received_cents,0) AS balance_cents
 FROM appointments a LEFT JOIN
 (SELECT appointment_id,SUM(amount_cents) AS received_cents FROM payments GROUP BY appointment_id) p
 ON p.appointment_id=a.id;
PRAGMA user_version = 1;
COMMIT;
```

Apêndice 2. 02_seed.sql

O arquivo no diretório database é a versão executável. Linhas longas têm quebra visual neste apêndice.

```
-- Dados FICTÍCIOS para apresentação. Execute apenas em banco vazio.
-- Agenda relativa à data de São Paulo (UTC-3): a demonstração continua útil em outras datas.
PRAGMA foreign_keys=ON;
BEGIN;
INSERT INTO tutors(id,name,phone,email,address) VALUES
 (1,'Marina Oliveira','(71) 99900-1001','marina@example.com','Salvador, BA'),
 (2,'Rafael Santos','(71) 99900-1002','rafael@example.com','Salvador, BA'),
 (3,'Camila Costa','(71) 99900-1003',NULL,'Lauro de Freitas, BA'),
 (4,'Lucas Almeida','(71) 99900-1004',NULL,'Salvador, BA'),
 (5,'Juliana Lima','(71) 99900-1005','juliana@example.com','Salvador, BA'),
 (6,'Bruno Rocha','(71) 99900-1006',NULL,'Salvador, BA');
INSERT INTO pets(id,tutor_id,name,species,breed,sex,birth_date,weight_kg,notes) VALUES
 (1,1,'Luna','Cão','Shih-tzu','Fêmea','2022-04-12',5.2,''),
 (2,2,'Thor','Cão','Golden retriever','Macho','2021-07-20',28.0,''),
 (3,3,'Mel','Gato','Sem raça definida','Fêmea','2023-01-10',3.8,''),
 (4,4,'Nino','Gato','Siamês','Macho','2020-11-05',4.5,''),
 (5,5,'Amora','Cão','Poodle','Fêmea','2024-03-15',4.0,''),
 (6,6,'Bento','Cão','Sem raça definida','Macho','2022-06-01',12.0,''),
 (7,1,'Pipoca','Ave','Calopsita','Não informado','2024-01-10',0.09,''),
 (8,3,'Olívia','Gato','Sem raça definida','Fêmea','2025-02-10',2.7,'');
INSERT INTO veterinarians(id,name,crmv,specialty,phone) VALUES
 (1,'Ana Martins','BA-DEMO-001','Clínica geral','(71) 99900-2001'),
 (2,'Felipe Souza','BA-DEMO-002','Dermatologia','(71) 99900-2002');
INSERT INTO services(id,name,price_cents,duration_minutes) VALUES
 (1,'Consulta clínica',18000,30),(2,'Vacinação',9000,20),
 (3,'Retorno',0,20),(4,'Consulta dermatológica',24000,45);
INSERT INTO appointments(id,pet_id,veterinarian_id,service_id,starts_at,duration_minutes,fee_cents,status,notes) VALUES
 (1,1,1,1,date('now','-3 hours')||'T08:30',30,18000,'Concluído',''),
 (2,2,1,2,date('now','-3 hours')||'T09:30',20,9000,'Confirmado',''),
 (3,3,2,4,date('now','-3 hours')||'T10:00',45,24000,'Agendado',''),
 (4,4,1,3,date('now','-3 hours')||'T11:00',20,0,'Agendado',''),
 (5,5,2,1,date('now','-3 hours')||'T14:00',30,18000,'Confirmado',''),
 (6,6,1,1,date('now','-3 hours')||'T15:30',30,18000,'Agendado',''),
 (7,8,1,1,date('now','-3 hours','+1 day')||'T09:00',30,18000,'Agendado',''),
 (8,7,2,1,date('now','-3 hours','+2 days')||'T10:30',30,18000,'Agendado',''),
 (9,2,1,1,date('now','-3 hours','-2 days')||'T10:00',30,18000,'Concluído','');
INSERT INTO medical_records(appointment_id,complaint,diagnosis,treatment,weight_kg) VALUES
 (1,'Avaliação de rotina','Registro fictício para demonstração','Orientações fictícias de acompanhamento',5.2),
 (9,'Avaliação de rotina','Registro fictício para demonstração','Retorno de acompanhamento',28.0);
INSERT INTO payments(appointment_id,amount_cents,method,paid_at) VALUES
 (1,18000,'Pix',date('now','-3 hours')),
 (9,18000,'Cartão',date('now','-3 hours','-2 days'));
COMMIT;
```

Apêndice 3. 03_queries.sql

O arquivo no diretório database é a versão executável. Linhas longas têm quebra visual neste apêndice.

```
-- Consultas didáticas: execute após 01_schema.sql e 02_seed.sql.
PRAGMA foreign_keys=ON;

-- Q1: agenda do dia, com JOIN entre as entidades do atendimento.
SELECT a.id,substr(a.starts_at,12,5) AS horario,p.name AS animal,
 t.name AS tutor,v.name AS veterinario,s.name AS servico,a.status
FROM appointments a
JOIN pets p ON p.id=a.pet_id
JOIN tutors t ON t.id=p.tutor_id
JOIN veterinarians v ON v.id=a.veterinarian_id
JOIN services s ON s.id=a.service_id
WHERE substr(a.starts_at,1,10)=date('now','-3 hours')
ORDER BY a.starts_at;

-- Q2: animais de cada tutor. LEFT JOIN inclui quem não tem animal cadastrado.
SELECT t.id,t.name,COUNT(p.id) AS animais_ativos
FROM tutors t LEFT JOIN pets p ON p.tutor_id=t.id AND p.active=1
GROUP BY t.id,t.name ORDER BY animais_ativos DESC,t.name;

-- Q3: receita por forma de pagamento no mês corrente.
-- Receita significa caixa recebido, não soma dos serviços agendados.
SELECT method,SUM(amount_cents) / 100.0 AS recebido_reais
FROM payments
WHERE substr(paid_at,1,7)=strftime('%Y-%m','now','-3 hours')
GROUP BY method ORDER BY recebido_reais DESC;

-- Q4: consultas concluídas com saldo. A view agrega pagamentos uma única vez.
SELECT a.id,p.name AS animal,t.name AS tutor,
 a.fee_cents / 100.0 AS valor_reais,
 a.received_cents / 100.0 AS recebido_reais,
 a.balance_cents / 100.0 AS saldo_reais
FROM appointment_balances a
JOIN pets p ON p.id=a.pet_id JOIN tutors t ON t.id=p.tutor_id
WHERE a.status='Concluído' AND a.balance_cents>0
ORDER BY a.starts_at;

-- Q5: histórico clínico. Troque 1 pelo ID desejado ao executar no SQLite.
SELECT a.starts_at,v.name AS veterinario,r.complaint,r.diagnosis,
 r.treatment,r.weight_kg
FROM medical_records r
JOIN appointments a ON a.id=r.appointment_id
JOIN veterinarians v ON v.id=a.veterinarian_id
WHERE a.pet_id=1 ORDER BY a.starts_at DESC;

-- Q6: volume de atendimentos concluídos por veterinário.
SELECT v.name,COUNT(a.id) AS atendimentos
FROM veterinarians v LEFT JOIN appointments a
 ON a.veterinarian_id=v.id AND a.status='Concluído'
GROUP BY v.id,v.name ORDER BY atendimentos DESC;

-- Q7: inspeção da integridade física e dos vínculos.
PRAGMA integrity_check;
PRAGMA foreign_key_check;

-- Q8: observe o índice usado na consulta de agenda por intervalo.
-- Range de datas aproveita idx_agenda_date, ao contrário de substr na coluna.
EXPLAIN QUERY PLAN SELECT * FROM appointments
WHERE starts_at >= date('now','-3 hours') || 'T00:00'
 AND starts_at < date('now','-3 hours','+1 day') || 'T00:00';
```

Apêndice 4. 04_transactions.sql

O arquivo no diretório database é a versão executável. Linhas longas têm quebra visual neste apêndice.

```
-- Exemplos de DML sem efeito permanente: cada transação termina em ROLLBACK.
-- Use um banco de demonstração. IDs negativos são reservados aqui para os exemplos.
PRAGMA foreign_keys=ON;

-- INSERT: cadastrar tutor e animal vinculados de forma atômica.
BEGIN IMMEDIATE;
INSERT INTO tutors(id,name,phone) VALUES(-1,'Tutor de exemplo','(71) 99999-0000');
INSERT INTO pets(id,tutor_id,name,species,sex) VALUES(-1,-1,'Animal exemplo','Cão','Macho');
SELECT p.name,t.name FROM pets p JOIN tutors t ON t.id=p.tutor_id WHERE p.id=-1;
ROLLBACK;

-- UPDATE: alterar preço do catálogo. A consulta antiga conserva fee_cents.
BEGIN;
UPDATE services SET price_cents=20000 WHERE id=1;
SELECT s.price_cents AS preco_atual,a.fee_cents AS preco_da_consulta
FROM services s JOIN appointments a ON a.service_id=s.id WHERE s.id=1;
ROLLBACK;

-- DELETE: remover apenas registros de exemplo sem dependências restantes.
-- A interface usa arquivamento para os cadastros da clínica.
BEGIN;
INSERT INTO tutors(id,name,phone) VALUES(-2,'Tutor temporário','(71) 99999-0001');
INSERT INTO pets(id,tutor_id,name,species) VALUES(-2,-2,'Animal temporário','Gato');
DELETE FROM pets WHERE id=-2;
DELETE FROM tutors WHERE id=-2;
ROLLBACK;

-- Na aplicação, concluir um atendimento executa UPDATE + INSERT sob a mesma
-- transação (api.py, finish_appointment). COMMIT ocorre só quando ambos passam.
-- Chaves estrangeiras, checks e triggers também atuam em DML executado fora da API.
```